DSA-II · DIVIDE AND CONQUER

PDF Variations in Sir's Class Style

Eight complete, separately runnable C++ programs using global vectors, start/end recursion, and the same Divide–Conquer–Merge comment pattern.

Prepared from every problem in the supplied *Divide and Conquer.pdf*.

Program Index

01
Merge Sort in Descending Order
PDF 1(a): base-case comparison and merge comparison are reversed so larger elements come first.

02
Three-Way Merge Sort
PDF 1(b): divide into three sections, recursively sort all three, then merge three sorted vectors.

03
Sum of All Even Numbers
PDF 2: a one-element subarray returns the element if it is even, otherwise 0; merge by addition.

04
Second Maximum Number
PDF 3: each recursive call returns (maximum, second maximum), then four candidates are combined.

05
Second Minimum Number
PDF 4: the second-maximum method is converted into a minimum version.

06
Calculate b^n
PDF 5: calculate b^(n/2) once, square it, and multiply one extra b when n is odd.

07
Count Multiples of 3 or 5
PDF 6: each leaf returns 1 or 0; merge by adding the left and right counts.

08
Minimum-Sum Subarray
PDF 7: the maximum-subarray class code is converted by using min and positive infinity.

How to practice: prothome comment dekhe function-ta nije likho; tarpor code-er shathe milao. Protita program alada file, karon protitar nijer `main()` ache.

PROGRAM 01

Merge Sort in Descending Order

PDF 1(a): base-case comparison and merge comparison are reversed so larger elements come first.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

void _merge_sort_descending(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(_end-start==1) /// size 2
    {
        if(v[start]<v[_end])
        {
            swap(v[start],v[_end]);
        }
        return;
    }
    else if(start==_end) /// size 1
        return;

    ///step-1: divide
    int mid=(start+_end)/2;
    _merge_sort_descending(start,mid); /// sort from start to mid
    _merge_sort_descending(mid+1,_end);/// sort from mid+1 to end


    ///step-3: merge
    vector<int>l,r; /// l=v[start...mid], r=v[mid+1..._end]
    for(int i=start;i<=mid;i++)
        l.push_back(v[i]);
    for(int i=mid+1;i<=_end;i++)
        r.push_back(v[i]);

    int i=0,j=0;
    vector<int>new_array;

    while(i<l.size() && j<r.size())
    {
        /// larger element will come first
        if(l[i]>r[j])
        {
            new_array.push_back(l[i]);
            i++;
        }
        else
        {
            new_array.push_back(r[j]);
            j++;
        }
    }

    while(i<l.size())
    {
        new_array.push_back(l[i]);
        i++;
    }

    while(j<r.size())
    {
```

```
        new_array.push_back(r[j]);
        j++;
    }

    /// copy new_array in our main vector v
    int p=0;
    for(int k=start;k<=_end;k++)
    {
        v[k]=new_array[p];
        p++;
    }
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>0)
        _merge_sort_descending(0,n-1);

    cout<<"after sorting in descending order-----------"<<endl;
    for(int i=0;i<n;i++)
        cout<<v[i]<<" ";
}

/// sample input
/*
9
-5 -10 16 2 6 5 3 2 19
*/

/// sample output
/*
19 16 6 5 3 2 2 -5 -10
*/
```

PROGRAM 02

Three-Way Merge Sort

PDF 1(b): divide into three sections, recursively sort all three, then merge three sorted vectors.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

void _merge_sort_3_way(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(_end-start==1) /// size 2
    {
        if(v[start]>v[_end])
        {
            swap(v[start],v[_end]);
        }
        return;
    }
    else if(start==_end) /// size 1
        return;

    ///step-1: divide the array into 3 sections
    int mid1=start+(_end-start)/3;
    int mid2=start+2*(_end-start)/3;

    _merge_sort_3_way(start,mid1);
    _merge_sort_3_way(mid1+1,mid2);
    _merge_sort_3_way(mid2+1,_end);


    ///step-3: merge 3 sorted arrays
    vector<int>l,m,r;

    for(int i=start;i<=mid1;i++)
        l.push_back(v[i]);
    for(int i=mid1+1;i<=mid2;i++)
        m.push_back(v[i]);
    for(int i=mid2+1;i<=_end;i++)
        r.push_back(v[i]);

    int i=0,j=0,k=0;
    vector<int>new_array;

    /// all three pointers are inside the bounds
    while(i<l.size() && j<m.size() && k<r.size())
    {
        if(l[i]<=m[j] && l[i]<=r[k])
        {
            new_array.push_back(l[i]);
            i++;
        }
        else if(m[j]<=l[i] && m[j]<=r[k])
        {
            new_array.push_back(m[j]);
            j++;
        }
        else
        {
            new_array.push_back(r[k]);
            k++;
```

```
        }
    }

    /// merge the two remaining arrays
    while(i<l.size() && j<m.size())
    {
        if(l[i]<=m[j]) new_array.push_back(l[i++]);
        else new_array.push_back(m[j++]);
    }

    while(i<l.size() && k<r.size())
    {
        if(l[i]<=r[k]) new_array.push_back(l[i++]);
        else new_array.push_back(r[k++]);
    }

    while(j<m.size() && k<r.size())
    {
        if(m[j]<=r[k]) new_array.push_back(m[j++]);
        else new_array.push_back(r[k++]);
    }

    /// copy the remaining elements
    while(i<l.size()) new_array.push_back(l[i++]);
    while(j<m.size()) new_array.push_back(m[j++]);
    while(k<r.size()) new_array.push_back(r[k++]);

    /// copy new_array in our main vector v
    int p=0;
    for(int index=start;index<=_end;index++)
    {
        v[index]=new_array[p];
        p++;
    }
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>0)
        _merge_sort_3_way(0,n-1);

    cout<<"after 3-way merge sorting----------"<<endl;
    for(int i=0;i<n;i++)
        cout<<v[i]<<" ";
}

/// sample input
/*
10
15 2 19 -5 7 3 3 20 1 8
*/

/// sample output
/*
-5 1 2 3 3 7 8 15 19 20
*/
```

PROGRAM 03

Sum of All Even Numbers

PDF 2: a one-element subarray returns the element if it is even, otherwise 0; merge by addition.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

int even_sum(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(start==_end) /// size 1
    {
        if(v[start]%2==0)
            return v[start];
        else
            return 0;
    }

    ///step-1: divide
    int mid=(start+_end)/2;
    int left_even_sum=even_sum(start,mid);
    int right_even_sum=even_sum(mid+1,_end);


    ///step-3: merge
    int merged_even_sum=left_even_sum+right_even_sum;
    return merged_even_sum;
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>0)
        cout<<even_sum(0,n-1)<<endl;
}

/// sample input
/*
6
5 8 3 12 7 10
*/

/// sample output
/*
30
*/
```

How the even numbers are added

Example: `[5, 8, 3, 12, 7, 10]`. A one-element call returns the number when it is even; an odd number returns 0. Returning from recursion adds the left and right answers.

```
even_sum(0,5)
├── even_sum(0,2)
│   ├── [5,8]  → 0 + 8 = 8
│   └── [3]    → 0
│   merged answer = 8
│
└── even_sum(3,5)
    ├── [12,7] → 12 + 0 = 12
    └── [10]   → 10
    merged answer = 22

Final answer = 8 + 22 = 30
```

So the calculation is: **0 + 8 + 0 + 12 + 0 + 10 = 30**.

PROGRAM 04

Second Maximum Number

PDF 3: each recursive call returns (maximum, second maximum), then four candidates are combined.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

/// the pair will store (maximum, second maximum)
pair<int,int> find_second_maximum(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(_end-start==1) /// size 2
    {
        if(v[start]>v[_end])
            return make_pair(v[start],v[_end]);
        else
            return make_pair(v[_end],v[start]);
    }
    else if(start==_end) /// size 1
    {
        return make_pair(v[start],-100000000);
    }

    ///step-1: divide
    int mid=(start+_end)/2;
    pair<int,int>left_answer=find_second_maximum(start,mid);
    pair<int,int>right_answer=find_second_maximum(mid+1,_end);


    ///step-3: merge
    /// candidates are max1, second_max1, max2, second_max2
    vector<int>candidate;
    candidate.push_back(left_answer.first);
    candidate.push_back(left_answer.second);
    candidate.push_back(right_answer.first);
    candidate.push_back(right_answer.second);

    int maximum=-100000000;
    int second_maximum=-100000000;

    for(int i=0;i<candidate.size();i++)
    {
        if(candidate[i]>maximum)
        {
            second_maximum=maximum;
            maximum=candidate[i];
        }
        else if(candidate[i]>second_maximum)
        {
            second_maximum=candidate[i];
        }
    }

    return make_pair(maximum,second_maximum);
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
```

```
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>=2)
    {
        pair<int,int>answer=find_second_maximum(0,n-1);
        cout<<"maximum = "<<answer.first<<endl;
        cout<<"second maximum = "<<answer.second<<endl;
    }
}

/// sample input
/*
5
7 2 9 4 6
*/

/// sample output
/*
maximum = 9
second maximum = 7
*/

/// Note: duplicates are counted as separate occurrences.
/// For 7 9 9, the maximum and second maximum are both 9.
```

PDF-e “distinct” bola nei, tai duplicate separate occurrence hisebe count kora hoyeche. Question-e “second distinct” bolle duplicate skip korte hobe.

PROGRAM 05

Second Minimum Number

PDF 4: the second-maximum method is converted into a minimum version.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

/// the pair will store (minimum, second minimum)
pair<int,int> find_second_minimum(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(_end-start==1) /// size 2
    {
        if(v[start]<v[_end])
            return make_pair(v[start],v[_end]);
        else
            return make_pair(v[_end],v[start]);
    }
    else if(start==_end) /// size 1
    {
        return make_pair(v[start],100000000);
    }

    ///step-1: divide
    int mid=(start+_end)/2;
    pair<int,int>left_answer=find_second_minimum(start,mid);
    pair<int,int>right_answer=find_second_minimum(mid+1,_end);


    ///step-3: merge
    /// candidates are min1, second_min1, min2, second_min2
    vector<int>candidate;
    candidate.push_back(left_answer.first);
    candidate.push_back(left_answer.second);
    candidate.push_back(right_answer.first);
    candidate.push_back(right_answer.second);

    int minimum=100000000;
    int second_minimum=100000000;

    for(int i=0;i<candidate.size();i++)
    {
        if(candidate[i]<minimum)
        {
            second_minimum=minimum;
            minimum=candidate[i];
        }
        else if(candidate[i]<second_minimum)
        {
            second_minimum=candidate[i];
        }
    }

    return make_pair(minimum,second_minimum);
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
```

```
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>=2)
    {
        pair<int,int>answer=find_second_minimum(0,n-1);
        cout<<"minimum = "<<answer.first<<endl;
        cout<<"second minimum = "<<answer.second<<endl;
    }
}

/// sample input
/*
5
7 2 9 4 6
*/

/// sample output
/*
minimum = 2
second minimum = 4
*/

/// Note: duplicates are counted as separate occurrences.
/// For 2 2 9, the minimum and second minimum are both 2.
```

PDF-e “distinct” bola nei, tai duplicate separate occurrence hisebe count kora hoyeche. Question-e “second distinct” bolle duplicate skip korte hobe.

PROGRAM 06

Calculate b^n

PDF 5: calculate b^(n/2) once, square it, and multiply one extra b when n is odd.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
using namespace std;
long long b;
int n;

long long calculate_power(int exponent)
{
    ///step-2: conquer (base condition)
    if(exponent==0)
        return 1;
    else if(exponent==1)
        return b;

    ///step-1: divide
    int half_exponent=exponent/2;
    long long half_answer=calculate_power(half_exponent);


    ///step-3: merge
    long long answer=half_answer*half_answer;

    /// if exponent is odd, multiply one extra b
    if(exponent%2==1)
        answer=answer*b;

    return answer;
}

int main()
{
    cin>>b>>n;
    cout<<calculate_power(n)<<endl;
}

/// sample input
/*
3 13
*/

/// sample output
/*
1594323
*/

/// Assumption: exponent n is non-negative and the answer fits in long long.
```

PROGRAM 07

Count Multiples of 3 or 5

PDF 6: each leaf returns 1 or 0; merge by adding the left and right counts.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

int count_multiple_3_or_5(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(start==_end) /// size 1
    {
        if(v[start]%3==0 || v[start]%5==0)
            return 1;
        else
            return 0;
    }

    ///step-1: divide
    int mid=(start+_end)/2;
    int left_count=count_multiple_3_or_5(start,mid);
    int right_count=count_multiple_3_or_5(mid+1,_end);


    ///step-3: merge
    int merged_count=left_count+right_count;
    return merged_count;
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>0)
        cout<<count_multiple_3_or_5(0,n-1)<<endl;
}

/// sample input
/*
7
3 5 7 9 10 11 15
*/

/// sample output
/*
5
*/

/// 15 is counted only once because the condition uses OR.
```

PROGRAM 08

Minimum-Sum Subarray

PDF 7: the maximum-subarray class code is converted by using min and positive infinity.

Class-style structure: step-2 conquer/base condition → step-1 divide/recursive calls → step-3 merge/combine.

```
#include<iostream>
#include<vector>
using namespace std;
int n;
vector<int>v;

int minsum_subarray(int start, int _end)
{
    ///step-2: conquer (base condition)
    if(start==_end) /// size 1
        return v[start];


    ///step-1: divide
    int mid=(start+_end)/2;
    int leftmin=minsum_subarray(start,mid);
    int rightmin=minsum_subarray(mid+1,_end);


    ///step-3: merge

    int lm=100000000; /// from mid -> start, tracker of minimum sum
    int current_sum=0;

    for(int i=mid;i>=start;i--)
    {
        current_sum+=v[i];
        lm=min(lm,current_sum);
    }

    int rm=100000000; /// from mid+1 -> end, tracker of minimum sum
    current_sum=0;

    for(int i=mid+1;i<=_end;i++)
    {
        current_sum+=v[i];
        rm=min(rm,current_sum);
    }

    int merged_min=lm+rm;
    int ans=min(merged_min,min(leftmin,rightmin));

    return ans;
}

int main()
{
    cin>>n;
    for(int i=0;i<n;i++)
    {
        int temp;
        cin>>temp;
        v.push_back(temp);
    }

    if(n>0)
    {
        int minsum=minsum_subarray(0,n-1);
        cout<<minsum<<endl;
```

```
    }
}

/// sample input from the PDF
/*
10
3 -2 5 -4 6 -1 -2 3 -5 2
*/

/// sample output
/*
-5
*/
```

PDF temperature input-er minimum sum = -5. Day 9 only ebong days 6–9—duita range-er sum-i -5.